

conditional probability for two points x_i and x_j in the higher dimension is given by:

$$P(j|i) = \exp(-\|x_i - x_j\|^2 / 2\sigma^2) / \sum_{k: i \neq k} \exp(-\|x_i - x_k\|^2 / 2\sigma^2)$$

...where σ^2 is the variance of the Gaussian distribution centred on datapoint x_i . The main idea behind the SNE algorithm is that two data points x_i and x_j , which are neighbours, will be close if P_{ji} is high and will be separate if P_{ji} is almost zero.

In the lower dimension, the conditional probability for the same two points y_i and y_j is given by:

$$Q(j|i) = \exp(-\|y_i - y_j\|^2) / \sum_{k: i \neq k} \exp(-\|y_i - y_k\|^2)$$

...where σ^2 is set to some constant like $1/\sqrt{2}$ to model the similarity of points y_i and y_j .

So, these two probabilities have to be equal to allow the similarities of data points in both the high and low dimensions. SNE achieves this by minimising the difference between these two distributions. But when the Gaussian distribution is used in SNE, there is a problem called the crowding problem. That is, if the data set has a huge number of data points that are closer in the higher dimension, then it tries to crowd them in a lower dimension. To solve this problem, the Student-t distribution was used instead of Gaussian distribution for a lower dimension, and hence the name t-SNE for this technique.

To achieve the similarities between the data points in the higher and lower dimensions using these two distributions, t-SNE uses the Kullback-Liebler (KL) divergence.

For two discrete distributions P and Q , KL divergence (DKL) is given by:

$$DKL(P \parallel Q) = \sum_i P_i (P_i / Q_i)$$

The t-SNE defines a cost function based on the difference between $P(j|i)$ and $Q(j|i)$, which is given by:

$$C = \sum_i \sum_j (P(j|i) \log(P(j|i) / Q(j|i)))$$

The algorithm uses the gradient update to minimise the cost function, so that the distributions in the higher and lower dimensions become similar.

Let us now try to apply t-SNE to the wine data, as done earlier for PCA and LDA:

```
from sklearn.manifold import TSNE
tsne = TSNE(random_state=21)
# use fit_transform instead of fit, as TSNE has no transform
method
trainX_tsne = tsne.fit_transform(x)
#print(len(trainX_tsne))
```

```
tsDf = pd.DataFrame(data = trainX_tsne
, columns = ['t-SNE feature 1', 't-SNE feature 2'])
tsneDf = pd.concat([tsDf, y], axis = 1)
```

```
fig = pyplot.figure(figsize = (7,7))
ax = fig.add_subplot(1,1,1)
ax.set_xlabel('t-SNE feature 1', fontsize = 15)
ax.set_ylabel('t-SNE feature 2', fontsize = 15)
ax.set_title('2 dimensional tsne', fontsize = 20)
targets = [0,1,2]
colors = ['r', 'g', 'b']
for target, color in zip(targets,colors)
    indicesToKeep = tsneDf['target'] == target
    ax.scatter(tsneDf.loc[indicesToKeep, 't-SNE feature 1']
, tsneDf.loc[indicesToKeep, 't-SNE feature 2']
, c = color
, s = 50)
ax.legend(targets)
ax.grid()
```

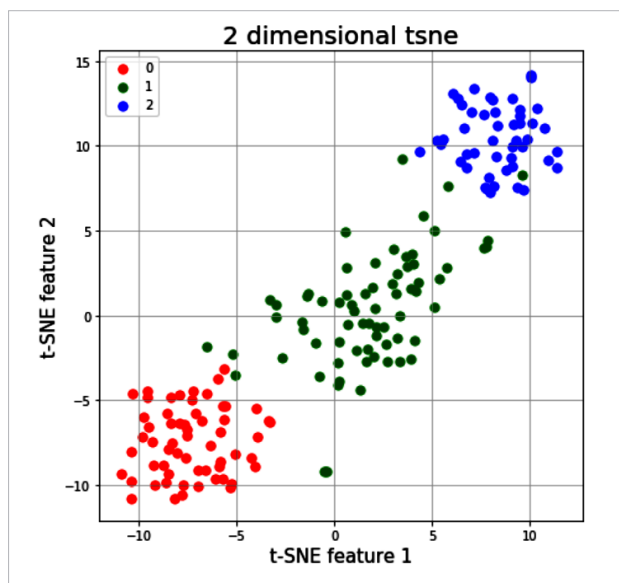


Figure 4: t-SNE for wine data set

The t-SNE technique is significantly useful when the number of input features is more — for example, as in the MNIST data set, in which each input sample is an image of size 28 x 28 pixels. So, each sample has around 784 features, and it will be very difficult to visualise the patterns of all 10 digits by using traditional clustering techniques. Given below is the code sample in which the t-SNE method is applied; it allows a user to visualise the data and its patterns in a very simple way:

```
import numpy as np
import pandas as pd
import seaborn as sns
```